Assignment 2 Appendix (Postgraduate)

Data Schemas, Examples, and Metrics

Authoritative source. Canvas provides the latest assignment files and updates.

Contents

1 Data Package and Input Schemas **2**
- 1.1 File layout . . . 2
- 1.2 Document schema (`documents.jsonl`) . . . 2
- 1.3 Question schema (`question.json`, `mini_dev.json`) . . . 3
- 1.4 Vector embedding store . . . 3
- 1.5 Embedding text construction . . . 4

2 Starter Functions and Saved Files **4**
- 2.1 Retrieval function . . . 4
- 2.2 Judging function . . . 5
- 2.3 Final CSV . . . 6

3 Baseline B0 mechanism **6**
- 3.1 Step 1: Boolean-style retrieval . . . 7
- 3.2 Step 2: Token-matching option judging . . . 7
- 3.3 Step 3: Aggregation . . . 7

4 Metrics **7**
- 4.1 Distribution score . . . 8
- 4.2 Evidence metric: MAP@100 . . . 9

1 Data Package and Input Schemas

1.1 File layout

The starter ZIP includes the mini data. Download the separate full data ZIP from Canvas and extract it into the starter folder. The full-data files should then appear under `data/full/` as listed below.

File	Purpose
`data/full/documents.jsonl`	Full corpus (one JSON object per line).
`data/full/question.json`	8 development and 10 hidden-test questions.
`data/full/document_embeddings.npy`	Precomputed dense document vectors.
`data/full/document_ids.json`	Document IDs in the same row order as the embedding matrix.
`data/full/embeddings_meta.json`	Embedding model, dimensions, text construction, and corpus checksum.
`data/mini/mini_documents.jsonl`	Mini corpus for the public self-check (1,000 documents).
`data/mini/mini_dev.json`	One mini question.
`data/mini/mini_groundtruth.json`	Mini reference distribution and relevant-document labels.
`data/mini/mini_document_embeddings.npy`	Supplied Float16 vectors with shape `(1000, 1536)`.
`data/mini/mini_document_ids.json`	Mini document IDs in vector row order.
`data/mini/mini_embeddings_meta.json`	Mini embedding model, dimensions and text construction.

Ethics statement. Treat retrieved documents as evidence, not verified truth or endorsement. The system estimates aggregate survey-style distributions and must not identify individuals. Questions and reference distributions use Pew Research Center materials. The cleaned public-web collection comes from SurveyPilot.

1.2 Document schema (`documents.jsonl`)

Each line in `documents.jsonl` is a UTF-8 JSON object with the following fields.

Field	Type	Notes
`id`	string	Unique opaque document identifier. Output this value in `supports`.
`url`	string	URL field stored with the document.
`source`	string	Platform or domain (e.g., `reddit.com`).
`group`	string	Sub-community or feed (e.g., a subreddit), useful for diversity and deduplication.
`title`	string	Page/post title, which may be empty.
`description`	string	Short summary if present, which may be empty.
`post_content`	string (optional)	Alternative body field present in some sources. Treat as empty if missing.
`content`	string	Main textual content.
`language`	string	Language tag (e.g., `"English"`).
`date`	string	Free-form date string. Parse it only if needed.

Example document


```
{"url":"https://example.invalid/thread/714280c72b101810",
 "source":"reddit.com",
 "title":"r/workingmoms on Reddit: How are your families finances handled?",
 "description":"I personally think it helps us be more transparent...",
 "language":"English",
 "content":"Fully joint accounts. My life would be...",
 "date":"October 23, 2024",
 "group":"r/workingmoms",
 "id":"VPWCKWQZME"}
```


1.3 Question schema (`question.json`, `mini_dev.json`)

`question.json` contains top-level `dev` and `test` objects. Each is keyed by the full question text. Each question value contains:

- `distribution`: an object mapping each exact option string to `0` in the supplied file.
- `supports`: an empty list in the supplied file.

The mini file maps question text directly to these objects, without split wrappers.

Example mini question entry


```
{
  "Which devices do you primarily use for gaming?": {
    "distribution": {
      "Mobile": 0,
      "Console": 0,
      "PC/Laptop": 0,
      "Tablet": 0,
      "VR Headset": 0
    },
    "supports": []
  }
}
```


Important: use each option string exactly as written.

1.4 Vector embedding store

File	Meaning
`document_embeddings.npy`	Float16 array with shape (183,303, 1536). Convert vectors to Float32 when scoring.
`document_ids.json`	JSON list of document-ID strings. Entry `ids[i]` identifies row `embeddings[i]`.
`embeddings_meta.json`	Metadata for validating the model, dimensions, text construction, and source corpus.

These files contain precomputed full-corpus document vectors. Build an ANN index if your method uses one.

```python
import json
from pathlib import Path
import numpy as np

root = Path("data/full")
ids = json.loads((root / "document_ids.json").read_text(encoding="utf-8"))
embeddings = np.load(root / "document_embeddings.npy", mmap_mode="r")
meta = json.loads((root / "embeddings_meta.json").read_text(encoding="utf-8"))
assert len(ids) == embeddings.shape[0]
```

In `retrieval/vectors.py`, `load_document_embeddings(config)` opens the mapped matrix and aligned IDs. `get_document_batch(...)` converts only the requested batch to Float32. `cosine_scores(...)` returns one cosine score per row.

1.5 Embedding text construction

Provided full-document embeddings. We used `text-embedding-3-small` to encode each document's title and content, joined by one newline. Empty titles are omitted:

```python
title = obj.get("title") or ""
content = obj.get("content") or ""
text = "\n".join(part for part in (title, content) if part)
```

Your query vectors. Use the same embedding model. You may vary the query text, for example by using the question alone or joining the question and exact answer options with newlines. The `embed_texts` helper in `api/embeddings.py` accepts a list of strings and returns their vectors.

Provided mini document vectors. The starter includes vectors for all 1,000 mini documents in `data/mini/`, with aligned IDs and model metadata. These use the same model and text construction as the full vectors. `mini_config.ini` points to the supplied files, ready for `load_document_embeddings(config)`.

2 Starter Functions and Saved Files

See the starter README for the file map. These functions receive and return Python objects.

Output folders. Run outputs are saved in the configured `run_dir`, such as `runs/mini_best/`. The default `cache/` folder holds generated indexes and raw API replies.

The retrieval and judging examples use B0's mini run for "Which devices do you primarily use for gaming?" The options are `Mobile`, `Console`, `PC/Laptop`, `Tablet` and `VR Headset`.

2.1 Retrieval function

File: `retrieval/rankers.py`

```python
def retrieve_and_rank(
    question: str, options: list[str], index: LexicalIndex,
    config: dict[str, Any], api_key: str | None = None,
) -> list[dict[str, Any]]:
    ...
```

Parameter	Meaning
`question`	Exact question text from the question file.
`options`	Exact answer-option strings in their original order.
`index`	Open `LexicalIndex` supplied by the runner.
`config`	Parsed INI settings, e.g. `config["retrieval"]["method"]`.
`api_key`	Course API key, or `None` when no API is needed.

Return. A list of exactly 100 dictionaries with unique corpus `doc_id` strings and finite numeric `score` values (`int` or `float`). Order by score, highest first. Equal scores are allowed. The first two records returned by B0 are:

```
[
    {"doc_id": "22E5PQJSLD", "score": 1.0},
    {"doc_id": "27PV7WSLUL", "score": 1.0}
]
```

Saved output. The retrieval command adds rank, text and metadata before saving `retrieved.jsonl`. Each line is a JSON object with `question`, `options` and the updated list under `ranked_documents`.

2.2 Judging function

File: `judging/judges.py`

```
def judge_evidence(
    question: str, options: list[str],
    ranked_documents: list[dict[str, Any]],
    config: dict[str, Any], api_key: str | None = None,
) -> list[dict[str, Any]]:
    ...
```

Parameter	Meaning
`question`	Exact question text from the question file.
`options`	Exact answer-option strings in their original order.
`ranked_documents`	The 100 records saved under `ranked_documents` in `retrieved.jsonl`, including text and metadata.
`config`	Parsed INI settings. `config["judging"]["judge_k"]` sets the number to judge, an integer from 1 to 100.
`api_key`	Course API key, or `None` when no API is needed.

The fourth record in `ranked_documents` from the mini run is shown below. `rank` is an integer starting at 1, `score` is numeric, and the other fields are strings. `text` joins title and content with a newline:

```
{
    "rank": 4,
    "doc_id": "2WZHJKU27B",
    "score": 1.0,
    "title": "r/gaming on Reddit: What are your thoughts on mobile gaming?",
    "text": "r/gaming on Reddit: What are your thoughts on mobile gaming?\nDepends on
    the game?",
    "source": "www.reddit.com",
    "group": "r/gaming",
    "url": "https://example.invalid/thread/78100665499186dd"
}
```

Return. A list of exactly `judge_k` dictionaries, one per selected document. Each contains a distinct `doc_id` string from `ranked_documents` and an `option` string matching an exact answer option or `non_relevant`. A short `reason` string is optional. The default `judge_k` is 20. For LLM judging, more documents usually increase API use. Two of B0's 20 returned records are:

```
[
    {"doc_id": "22E5PQJSLD", "option": "non_relevant",
     "reason": "No overlap"},
    {"doc_id": "2WZHJKU27B", "option": "Mobile",
     "reason": "B0 token overlap"}
]
```

The supplied `call_judging_model` helper returns the model's response as a string. Your judging function interprets that text and returns records in the format above.

Saved output. The judging command writes `judgments.jsonl` with one JSON object per question. Each object contains `question`, `options` and the returned list under `judgments`.

2.3 Final CSV

The judging command aggregates judgments into option probabilities, then writes these probabilities and the ranked document IDs to `submission.csv`. Kaggle expects one row per question, with `distribution` and `supports` encoded as JSON strings. Use exactly these three columns:

Column	Meaning
`question`	Full question string exactly as in the supplied question file.
`distribution`	JSON object mapping exact option strings to probabilities calculated from the selected judgments.
`supports`	JSON array of all 100 unique document IDs returned by retrieval, in the same ranked order.

Mini vs full. Mini uses the same schema with fewer question rows. Use mini CSVs for the public self-check and full CSVs for Kaggle.

Example mini CSV row. This example uses illustrative probabilities and the first three IDs from B0's ranked supports. The saved CSV contains all 100 IDs without an ellipsis.

```
question,distribution,supports
Which devices do you primarily use for gaming?,"{""Mobile"":0.30, ""Console"":0.25,
    ""PC/Laptop"":0.20, ""Tablet"":0.15, ""VR Headset"":0.10}","[""22E5PQJSLD"",
    ""27PV7WSLUL"", ""2C4DC3WSEP"" ... 97 more ...]"
```

Formatting tip. Use `csv.DictWriter` with `json.dumps()` to handle CSV quoting.

3 Baseline B0 mechanism

The unchanged baseline uses `mini_b0_config.ini` and `full_b0_config.ini`. Consider this question and its options:

> **Q:** "What is the most critical component of a perfect sandwich?"
> Options: *High-quality meat*, *Airy bread*, *Using all the right condiments*, *Fresh veggies*.

3.1 Step 1: Boolean-style retrieval

- **Tokenisation.** B0 uses the same Whoosh `StandardAnalyzer` for indexing, query processing, and option matching. You may use another permitted tokenizer in your own method, provided you apply it consistently.
- **OR matching.** B0 uses Boolean OR to retrieve documents containing any question or answer-option term. For example, a document mentioning "bread", "meat", "condiments", or "veggies" may match.
- **Result order.** B0 takes matches in document-ID order and fills any shortfall with other corpus IDs to return 100 supports. All receive a score of 1.0, so B0 does not rank them by relevance.

3.2 Step 2: Token-matching option judging

The B0 configs set `judge_k` to 20, so B0 judges the first 20 ranked supports. It assigns the option with the strongest token overlap, choosing the first option when positive overlaps tie. If all overlaps are zero, B0 returns `non_relevant`, which contributes no vote during aggregation.

Doc ID	Example text	Token-match signal	B0 option signal
RGPCNPLRDW	"... soft, airy bread makes or breaks it ..."	matches *bread*, *airy*	Airy bread
2QKWLQKGJ5	"... condiments such as mayo, mustard, and hot sauce are key ..."	matches *condiments*	Using all the right condiments
Y6X9L784EF	"... fresh tomato and crisp lettuce add the best texture ..."	matches *fresh*	Fresh veggies
QYUQZHDS73	"... if the meat is low quality the sandwich is ruined ..."	matches *meat*, *quality*	High-quality meat
TNGRQW9DMK	"... condiments balance salt and fat. Don't skip them ..."	matches *condiments*	Using all the right condiments
22E5PQJSLD	"... the meeting starts at nine tomorrow ..."	no option-token overlap	Abstain (`non_relevant`)

3.3 Step 3: Aggregation

Each non-abstaining document contributes one vote. The counts are meat 1, bread 1, condiments 2 and veggies 1. Normalising gives [0.20, 0.20, 0.40, 0.20] in that order. Abstentions contribute no votes. If every judgment abstains, B0 returns a uniform distribution.

4 Metrics

The starter implements the distribution score in `metrics/distribution.py` and MAP@100 in `metrics/evaluate.py`. You can read these files alongside the formulas below.

4.1 Distribution score

Intuition. A good prediction should estimate each option's probability accurately and identify which options are more likely.

The workshop used $1 - \text{JS}$ to measure distribution similarity. However, a uniform or heavily tied prediction can still receive a reasonable score even when the reference clearly favours some options. Our metric therefore combines:

- **distribution similarity (D):** how closely the full probability distribution matches the reference.
- **rank recovery (R):** how well the prediction recovers the order and size of gaps between meaningfully different options.

Let P be the reference distribution and Q the submitted distribution over the same options. Both are normalised. Add $\epsilon = 10^{-6}$ to each value and renormalise to obtain $\widetilde{P}$ and $\widetilde{Q}$. With base-2 logarithms:

$$M = \tfrac{1}{2}(\widetilde{P} + \widetilde{Q}), \qquad \text{JS}(\widetilde{P}, \widetilde{Q}) = \tfrac{1}{2}\,\text{KL}(\widetilde{P} \| M) + \tfrac{1}{2}\,\text{KL}(\widetilde{Q} \| M).$$

Distribution similarity is $D = 1 - \sqrt{\text{JS}(\widetilde{P}, \widetilde{Q})}$.

Compare option pairs. The rank component uses option pairs whose reference probabilities differ by more than $\tau = 0.02$ (two percentage points). These pairs form $\mathcal{I}$.

For options i and j, P_i and P_j are the reference probabilities, and Q_i and Q_j are the predicted probabilities. The reference gap is $g = P_i - P_j$, and the predicted gap is $h = Q_i - Q_j$.

Score each pair. A correct ordering adds credit, a reversed ordering subtracts credit, and a predicted tie contributes zero:

$$c_{ij} = \begin{cases} +\min(|g|, |h|), & g \times h > 0, \\ -\min(|g|, |h|), & g \times h < 0, \\ 0, & h = 0. \end{cases}$$

Each contribution uses the smaller absolute gap. For a reference gap of 0.20, a predicted gap of 0.10 contributes $+0.10$, a gap of -0.10 contributes -0.10, and a tie contributes 0. Exaggerating the predicted gap earns no extra credit.

Combine the pairs. Sum their contributions and divide by the total reference gap:

$$r = \frac{\sum_{(i,j)\in\mathcal{I}} c_{ij}}{\sum_{(i,j)\in\mathcal{I}} |g_{ij}|}, \qquad R = \max(0, r).$$

Recovering every reference gap in the correct direction gives $r = 1$. Correct and reversed pairs can offset each other. R replaces a negative total with zero, so $0 \le R \le 1$.

Combine distribution and rank. If $\mathcal{I}$ is empty, there are no sufficiently different options to rank, so the question score is $S_q = D$. Otherwise:

$$S_q = \frac{D(3R + 0.1)}{2R + D + 0.1}.$$

A high score requires both D and R to be high. The 0.1 term preserves a small amount of credit for distribution similarity when $R = 0$. For example, $D = 0.80$ and $R = 0.40$ give $S_q \approx 0.612$.

The final distribution score is $S = \text{mean}_q(S_q)$, the average across questions. A perfect prediction receives 1.

4.2 Evidence metric: MAP@100

We calculate average precision over the 100 submitted supports for each question, rather than the full ranked list used in lecture examples. This cutoff matches the required output size and gives every system the same evaluation depth.

For a ranked support list and relevant-document set G_q, average precision at cutoff $K = 100$ is:

$$\text{AP@K}(q) = \frac{1}{\min(|G_q|, K)} \sum_{k=1}^{K} \text{Precision@}\, k(q)\, \mathbf{1}[d_k \in G_q].$$

MAP@100 is the mean AP@100 across scored questions. It rewards relevant evidence appearing early in the ranked list. Support IDs must be unique and belong to the supplied corpus. The starter checks these requirements.